

Refleksjonsnotat – MatteHjelpen

ING100, KI-modulen · Innlevering 1 · Del C Utkast til gruppen: les gjennom, endre der dere mener noe annet, og fyll inn navnene deres. Dere skal kunne stå inne for hver påstand her.

1. Hva vi har bygget, og hva den faktisk kan brukes til

MatteHjelpen tar en matteoppgave som tekst, lar en språkmodell tolke oppgaven og velge metode, lar SymPy gjøre selve regningen gjennom verktøykall, og prøver til slutt å motbevise svaret med en uavhengig numerisk kontroll. Grensesnittet viser fire ting ved siden av svaret: hvordan oppgaven ble tolket, stegene med formelreferanser, hvilke verktøykall som *faktisk* ble utført, og hva kontrollen kom fram til.

De valgene vi er mest fornøyde med, er de som gjør appen *etterprøvbar* heller enn imponerende:

- Verktøyloggen bygges fra de faktiske `tool_calls`, ikke fra det modellen skriver om seg selv. Påstår modellen at den har brukt SymPy uten å ha gjort det, sier appen fra.
- Valideringen har tre utfall, ikke to: kontrollert, kontroll slo feil, og *ikke mulig å verifisere*. Et bevis havner i den siste kategorien.
- Tolkningen står ved siden av svaret, fordi kontrollen bare kan si noe om oppgaven slik modellen forstod den.

Bruksområdet er dermed smalt, med vilje: appen er et *kontrollerbart* hjelpemiddel for å forstå framgangsmåten i en oppgave man allerede jobber med. Den er ikke en fasit, og ikke et verktøy for å levere noe man ikke forstår.

2. Hjelpemiddel eller juks? Parallellen til kalkulatoren

Kalkulatoren tok fra oss regnearbeidet, men ikke ansvaret for oppsettet: vi må fortsatt vite hvilken utregning vi ber om, og om svaret er rimelig. Samme skille gjelder her. MatteHjelpen kan ta regnearbeidet, men ikke valget av metode, tolkningen av oppgaven eller vurderingen av om svaret gir mening.

Forskjellen fra kalkulatoren er at MatteHjelpen gjør ett steg til: den *tolker* oppgaven. En kalkulator som får $2+2$ regner alltid på $2+2$. En språkmodell som får «deriver $\sin^{-1}(x)$ » velger selv om det betyr arcsin eller $1/\sin$ – og den velger uten å spørre. Derfor er appen et hjelpemiddel så lenge man leser tolkningen og stegene, og juks i det øyeblikket man limer svaret rett inn i en innlevering. Den grensen går ikke ved teknologien, men ved hva man selv kan gjøre rede for.

3. Hva må vi kunne for å oppdage at appen tar feil?

Funnene i Del B gjorde dette konkret. På de ti pensumoppgavene svarte modellene riktig på nesten alt – også i kjøringene uten verktøy. Det var først da vi la til fire tyngre oppgaver at forskjellen kom fram. På det eksakte produktet $87\,654\,321 \times 12\,345\,679$ svarte den største modellen $1\,082\,152\,022\,374\,659$ uten verktøy. Riktig svar er $1\,082\,152\,110\,028\,959$. Feilen er på nesten 88 millioner, men svaret ser helt rimelig ut: riktig antall siffer, riktige første siffer, ingen tegn til at noe er galt. Med verktøy kalte den samme modellen `calculate` og fikk eksakt riktig svar.

Det er dette vi tar med oss: språkmodellen feilet ikke der den så usikker ut. Den feilet midt i et svar som så like trygt ut som alle de andre.

Vi ser tre nivåer av kontroll, og bare det første kan automatiseres:

- **Regnefeil** fanges av verktøy og validering. Da SymPy gjorde regningen, var svaret riktig i alle kjøringene der verktøyet faktisk ble brukt.
- **Tolkningsfeil** fanges ikke av noen automatisk kontroll. Da vi ga appen « $\text{deriver } \sin^{-1}(x)$ », nektet verktøyet vårt å gjette: parseren avviser den tvetydige notasjonen. Modellen svarte at notasjonen kan bety både $\arcsin(x)$ og $1/\sin(x)$, valgte $\arcsin$, regnet riktig og fikk grønt lys. Hadde vi ment $1/\sin(x)$, ville lyset vært like grønt. Valideringen kan bare svare på om svaret passer med det problemet modellen valgte å løse.
- **Relevansfeil** – om det i det hele tatt var riktig oppgave å løse – ligger helt utenfor appens rekkevidde.

For å oppdage feil på nivå 2 og 3 må vi kunne faget: vite at $\arcsin$ og $1/\sin$ er ulike funksjoner, at en 2. ordens differensialligning skal ha to vilkårlige konstanter, og ha en formening om hva svaret bør bli. Appen gjør oss ikke uavhengige av matematikken – den flytter kompetansekravet fra utregning til kontroll.

4. Kan vi stole på svarene?

Tallene fra Del B ser gode ut: modellen vi fikk kjørt hele rutenettet på, svarte riktig på alle de ni maskinsjekkable oppgavene både med og uten verktøy, og appen kontrollerte 38 av 43 kjøring. Den tiende oppgaven var et bevis, som appen ærlig merket som «ikke verifisert».

Vi må også si fra om vårt eget datagrunnlag: 17 av 60 celler mangler, fordi Groq først gikk tom for døgnkvote og siden sluttet å svare fra nettet vårt. Kravet på 40 kjøring er oppfylt, men sammenligningen «med mot uten verktøy» hviler i praksis på én modell. Det er en begrensning ved eksperimentet, ikke ved appen – og den hører hjemme her, ikke i en fotnote.

Vår konklusjon er at tallene *underdriver* risikoen. Oppgavene våre er hentet fra pensum, entydig formulert og med svar SymPy kan regne ut. I virkeligheten kommer oppgaver med uklar notasjon og forutsetninger som ikke står i teksten. Og selv her fant vi tilfeller der appen sa «validert» om svar på feil problem.

Noe mindre åpenbart: appen kontrollerer at formel-ID-ene finnes, men ikke at formelen er *brukt riktig*. I én kjøring oppga modellen kvotientregelen i en oppgave som bare krevde produkt- og kjerneregelen. ID-en var gyldig og referansen ekte. En gyldig referanse er ikke et kvalitetsstempel.

Det mest ubehagelige fant vi til slutt, da vi gikk løs på vår egen app for å ødelegge den: **kontrollen vår var selv feil**. Toleransen var rent relativ, så på produktet over godtok appen et avvik på over ti millioner – et svar som var *ett* for lite ble meldt som «kontrollert og stemmer». Verre: fasitsjekken vi måler modellene med i Del B, hadde samme hull, og kunne altså ikke ha avslørt feilen. Tallene holdt seg da vi regnet dem om, men det visste vi ikke før vi målte. Spørsmålet «hvor galt kan et svar være før den sier fra?» må stilles til ethvert verktøy som påstår å kontrollere noe – også til det man kontrollerer med.

5. Ville vi dimensjonert en bro med dette?

Nei – men det interessante er hvorfor ikke.

Ikke fordi regningen er upålitelig – SymPy regner bedre enn oss. Grunnen er at brodimensjonering ikke er et regnestykke, men en kjede: lastmodell → beregning → kontroll → beslutning. MatteHjelpen dekker bare det midterste leddet, og det var aldri det svakeste. Tay-broen falt ikke fordi noen regnet feil, men fordi vindlast manglet i modellen og kontrollen sviktet.

Å bruke appen til noe slikt ville betydd å la den velge premissene – altså akkurat det den ikke kan kontrolleres på. Vi ville derimot uten videre brukt den til å kontrollere en utregning vi selv hadde satt opp, og til å forstå en metode vi holder på å lære. Skillet går mellom «regn ut dette for meg» og «bestem hva som skal regnes ut».

6. Kan vi ta ansvar for koden?

Dette er spørsmålet vi synes er vanskeligst, fordi det meste av koden er skrevet av en KI-assistent (dokumentert i `KI-BRUK.md`). Vi kan forklare hva hver modul gjør og hvorfor de henger sammen som de gjør, men vi ville brukt tid på å forsvare parsing-modulen linje for linje.

Det som gjorde forskjellen for tilliten vår, var ikke lesing, men testing. Tjuetre ganger ga KI-assistenten oss kode som så riktig ut og var feil (se `KI-BRUK.md`): en parser-innstilling som stille gjorde $y(x)$ om til $y \cdot x$ og ødela alle differensialligninger, en sikkerhetssjekk som selv regnet ut det gigantiske tallet den skulle stoppe, en validator som ga opp på helt riktige svar – og, i den siste runden, en toleranse som godtok avvik på ti millioner og en sjekk som kunne låse appen for godt med ett enkelt uttrykk. Én eneste av de tjuetre ble funnet ved lesing.

Det gir svaret på spørsmålet i overskriften: vi kan ta ansvar for koden i den grad vi klarer å *avhøre* den – ta en påstand den gjør, og finne en måte å motbevise den på. Der vi bare har lest, har vi ikke ansvar, men en formening.

Hvis appen skulle gjøres tilgjengelig for andre studenter på nettet, ville vi måtte håndtere minst dette:

- *Sikkerhet*: SymPy tolker tekst ved å kjøre Python, og oppgaveteksten kommer fra brukeren. Vi fjernet innebygde funksjoner fra navnerommet, avviste `__`-navn og metodekall, og la hvert verktøykall i en egen prosess med tids- og minnegrense. Før publisering måtte dette vært gjennomgått av noen med mer kompetanse enn oss.
- *Kostnad og misbruk*: i dag betaler ingen, fordi vi bruker gratisnivåer. Med $1000 \text{ studenter} \times 50 \text{ oppgaver}$ blir listeprisen 620–4 490 kroner – og uten innlogging kunne hvem som helst brukt nøkkelen vår til hva som helst.
- *Ansvar for svarene*: en app som gir feil svar til én person i gruppa er en lærepeng. En app hundrevis av studenter bruker som fasit, er noe annet. Da må det være tydelig hvem som svarer for feilene, og grensesnittet må forklare hva «validert» faktisk betyr.

7. Hva vil det si å være flink i matte nå?

Vår konklusjon er at «flink i matte» flytter seg fra å *utføre* til å *spesifisere, kontrollere og begrunne*. Det betyr ikke at regneferdighet er verdiløs – tvert imot: det var nettopp det å kunne regne oppgave 4 for hånd som gjorde at vi kunne stole på kontrollen. Man må kunne nok matematikk til å vite når svaret er rimelig, og nok om verktøyet til å vite hva det ikke sjekker.

Det mest nyttige vi sitter igjen med, er ikke appen, men vanen: å spørre «hva ble faktisk kontrollert her?» før vi bruker et svar. Den vanen er den samme enten svaret kommer fra en språkmodell, et regneark eller en medstudent.

Notatet er skrevet med KI-hjelp på grunnlag av gruppens egne kjøringar og funn, se `KI-BRUK.md` for full dokumentasjon av KI-bruken.